Git and DevOps -- Practical Viva Questions and Answers

A beginner-friendly viva preparation guide covering Git, GitHub, version control, DevOps, CI/CD, Jenkins, Docker, Kubernetes, practical workflows, and important commands.

> ***How to use this guide:*** *Read each question first, try answering it*
>
> *in your own words, and then check the answer. The answers are written*
>
> *in simple language suitable for a practical viva.*

Part 1: Git and GitHub -- Viva Questions

1. What is Git?

Git is a **distributed version control system** used to track changes in source code. It helps developers save different versions of a project, work together, and return to an earlier version when required.

2. What is the difference between Git and GitHub?

- **Git** is a version control tool that runs on a computer and tracks

changes in files.

- **GitHub** is an online platform that hosts Git repositories and

supports collaboration through pull requests, issues, and code reviews.

In short, Git manages versions; GitHub hosts and helps people collaborate on Git projects.

3. Why do we use version control?

Version control helps us: - Track changes made to a project. - Identify who changed a file and when. - Restore an earlier version if something goes wrong. - Allow multiple developers to work on the same project. - Maintain a history of project development.

4. What is a repository?

A repository, or **repo**, is a storage location for a project's files and their version history. It contains the project and the information Git needs to track changes.

5. What is the difference between a local repository and a remote repository?

- **Local repository:** The Git repository stored on your own computer.
- **Remote repository:** A repository hosted on another computer or a service such as GitHub.

Developers usually make and commit changes locally, then push them to a remote repository to share them.

6. What is the purpose of the `git init` command?

`git init` initializes a new Git repository in the current folder. It creates a hidden `.git` directory where Git stores version-control information.

```
git init
```

7. What is the use of `git clone`?

`git clone` downloads an existing remote repository, including its files and Git history, to your computer.

```
git clone https://github.com/username/project-name.git
```

8. What is the difference between `git add` and `git commit`?

- `git add` moves selected changes to the **staging area**, preparing them for a commit.
- `git commit` saves the staged changes as a snapshot in the repository's history.

```
git add .
git commit -m "Add project files"
```

9. What does `git status` do?

`git status` displays the current state of the working directory and staging area. It tells us which files are modified, staged, untracked, or ready to commit.

```
git status
```

10. What is a branch in Git?

A branch is an independent line of development. It allows a developer to work on a feature or fix without directly changing the main branch.

11. How do you create a new branch?

Use `git branch` to create a branch:

```
git branch feature-login
```

To create and switch to it in one command:

```
git switch -c feature-login
```

12. What is the difference between `git merge` and `git rebase`?

- **Merge:** Combines changes from one branch into another and

generally creates a merge commit when needed. It preserves the branch history.

- **Rebase:** Replays commits from one branch on top of another,

creating a more linear history.

Use merge when preserving the branching history is important. Rebase can make history cleaner, but avoid rebasing commits that other people are already using because it rewrites commit history.

13. What is a merge conflict, and how do you resolve it?

A merge conflict occurs when Git cannot automatically combine changes, often because two branches changed the same part of a file.

To resolve it: 1. Run `git status` to identify conflicted files. 2. Open each file and inspect the conflict markers. 3. Edit the file to keep the correct final content. 4. Remove the conflict markers. 5. Stage the resolved file using `git add`. 6. Complete the merge with `git commit` if Git requests it.

14. What is the purpose of `git push` and `git pull`?

- `git push` uploads local commits to a remote repository.
- `git pull` downloads remote changes and integrates them into the

current local branch. It is broadly equivalent to `git fetch` followed by integration, usually a merge.

```
git push
git pull
```

15. What is a pull request?

A pull request (PR) is a request to merge changes from one branch into another, usually on GitHub. It lets team members review the code, discuss changes, suggest improvements, and approve the work before merging.

16. What is the difference between `git fetch` and `git pull`?

- `git fetch` downloads new commits and references from the remote repository but does not integrate them into the current branch.
- `git pull` fetches those changes and then integrates them into the current branch.

Use `git fetch` when you want to inspect remote changes before integrating them.

17. What is `.gitignore`?

`.gitignore` is a text file that tells Git which untracked files or folders it should ignore. It is commonly used for temporary files, build output, virtual environments, and local configuration files.

Example:

```
__pycache__/
.venv/
.env
*.log
```

Important: `.gitignore` does not automatically stop tracking a file that Git is already tracking.

18. How can you check the commit history in Git?

Use `git log` to view commit history, including commit IDs, authors, dates, and messages.

```
git log
```

A compact view can be shown with:

```
git log --oneline
```

19. What is the use of `git checkout` or `git switch`?

- `git switch` is the modern command mainly used to switch branches or create and switch to a branch.
- `git checkout` is an older, multi-purpose command that can switch branches and restore files.

Examples:

```
git switch main
git switch -c feature
```

20. How do you undo a commit in Git?

The correct method depends on whether the commit has been shared and whether you want to preserve the changes.

• `git revert <commit-id>` creates a new commit that reverses the changes. This is usually the safer choice for commits already pushed and shared.

• `git reset --soft HEAD~1` removes the latest commit but keeps its changes staged.

• `git reset --hard HEAD~1` removes the latest commit and its changes from the working tree. It can cause data loss, so use it carefully.

Part 2: DevOps -- Viva Questions

21. What is DevOps?

DevOps is a set of practices and a working culture that brings development (Dev) and operations (Ops) teams together. Its goal is to build, test, release, and maintain software more efficiently through collaboration and automation.

22. Why is DevOps important in software development?

DevOps helps teams: - Release software faster and more frequently. - Automate repetitive tasks. - Find and fix problems earlier. - Improve collaboration between development and operations. - Make deployments more consistent and reliable.

23. What is the difference between the Development and Operations teams?

• **Development team:** Designs, writes, and tests application code and features.

• **Operations team:** Manages infrastructure, deployment, availability, monitoring, and reliability.

DevOps encourages these teams to collaborate and share responsibility across the software lifecycle.

24. What is the DevOps lifecycle?

The DevOps lifecycle is a continuous process commonly represented by these stages:

1. **Plan:** Decide requirements and tasks.
2. **Code:** Develop the application.
3. **Build:** Compile or package the code.
4. **Test:** Check quality and find defects.
5. **Release:** Prepare a version for release.

6. **Deploy:** Make the application available in an environment.
7. **Operate:** Run and maintain the application.
8. **Monitor:** Track performance, errors, and availability.

Feedback from monitoring helps teams plan improvements, so the lifecycle repeats.

25. What is Continuous Integration (CI)?

Continuous Integration is the practice of frequently merging code changes into a shared repository. Automated builds and tests run to detect integration problems early.

26. What is Continuous Delivery (CD)?

Continuous Delivery is a practice in which code changes are automatically built, tested, and prepared for release. The application is kept ready to deploy, but production deployment may require manual approval.

27. What is the difference between Continuous Delivery and Continuous Deployment?

- **Continuous Delivery:** Every change that passes the pipeline is

ready for production, but a person may approve the production release.

- **Continuous Deployment:** Every change that passes all required

checks is automatically deployed to production without a manual release approval.

28. What is a CI/CD pipeline?

A CI/CD pipeline is an automated sequence of steps that takes code from a developer's changes through building, testing, and delivery or deployment.

A simple pipeline is:

```
Code → Build → Test → Release/Deploy
```

It reduces manual work and helps make software releases repeatable.

29. What is Jenkins?

Jenkins is an open-source automation server used to automate tasks such as building, testing, and deploying software. It can connect to source-control systems and run CI/CD pipelines using jobs or pipeline scripts.

30. What is Docker?

Docker is a platform used to build, package, and run applications in containers. It packages an application with many of its dependencies so that it can run more consistently across different environments.

31. What is a container?

A container is an isolated running environment for an application and its required files, libraries, and dependencies. Containers share the host operating-system kernel, making them generally lighter than virtual machines.

32. What is the difference between a virtual machine and a container?

- **Virtual machine (VM):** Virtualizes hardware and runs its own

guest operating system.

- **Container:** Isolates an application and its dependencies while

sharing the host OS kernel.

Containers generally start faster and use fewer resources. VMs provide a separate operating-system environment and stronger isolation boundaries in many setups.

33. What is Kubernetes?

Kubernetes is an open-source container orchestration platform. It helps deploy, scale, manage, and recover containerized applications across a cluster of machines.

Part 3: Git and DevOps -- Practical Viva Questions

34. How do you upload a project from your computer to GitHub?

One common method is:

1. Create an empty repository on GitHub.
2. Open a terminal in your project folder.
3. Initialize Git if the folder is not already a repository.
4. Stage and commit the project files.
5. Add the GitHub repository as a remote.
6. Push the branch to GitHub.

Commands:

```
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/username/project-name.git
git push -u origin main
```

Replace the URL with your own GitHub repository URL. If the folder is already a Git repository or already has a remote, check its configuration before repeating the initialization or remote steps.

35. How do you create and switch to a new branch?

Use:

```
git switch -c feature-name
```

This creates the branch and switches to it. To see all local branches, use `git branch`.

36. How do multiple developers work on the same project using Git?

A typical team workflow is:

1. Each developer clones the shared repository.
2. Each developer creates a separate branch for a feature or bug fix.
3. Developers make changes and commit them locally.
4. They push their branches to the remote repository.
5. They create pull requests for code review.
6. After review and approval, the changes are merged into the main branch.
7. Developers pull the latest changes before starting further work.

This workflow reduces direct conflicts in the main branch and supports code review.

37. How do you resolve a merge conflict?

1. Attempt the merge or pull.
2. Run `git status` to find conflicted files.
3. Open the files and inspect sections between `<<<<<<<`, `=======`, and `>>>>>>>`.
4. Decide which changes to keep, or combine them.
5. Remove the conflict markers and save the files.
6. Stage the resolved files with `git add`.
7. Complete the merge with `git commit` if required.

Always test the application after resolving a conflict.

38. How can Git be integrated with Jenkins?

Jenkins can connect to a Git repository using its Git-related plugins and repository configuration. A typical setup is:

1. Store the project in GitHub or another Git server.

2. Configure a Jenkins job or pipeline with the repository URL.
3. Configure credentials if the repository is private.
4. Set a trigger, such as a webhook or periodic polling.
5. When a change is detected, Jenkins checks out the code and runs the configured build and test steps.

39. How does a CI/CD pipeline deploy an application?

A typical pipeline works like this:

1. A developer pushes code to the repository.
2. The CI server detects the change.
3. The pipeline downloads the code.
4. It builds the application.
5. Automated tests and quality checks run.
6. If the checks pass, the application is packaged or an image is created.
7. The pipeline deploys it to a test or production environment, according to the configured approvals and rules.
8. Monitoring checks the deployed application.

If a required step fails, the pipeline normally stops and reports the failure.

40. How do Docker containers help in deployment?

Docker containers package an application and its dependencies into a consistent unit. This helps reduce differences between development, testing, and production environments. Containers can also be started, stopped, copied, and scaled more easily than manually configuring each environment.

Part 4: Important Git Commands

---------- Command Purpose ----------
---------- `git init` Initialize a Git repository

`git clone URL` Copy a remote repository to your computer

`git status` Check the working-tree and staging status

`git add .` Stage changes in the current directory

`git commit -m "message"` Save staged changes as a commit

`git branch` List local branches

`git branch name` Create a branch

`git switch branch-name` Switch to an existing branch

`git switch -c branch-name` Create and switch to a branch

`git merge branch-name` Merge the named branch into the current branch

`git push` Upload local commits to a remote

`git pull` Fetch and integrate remote changes

`git fetch` Download remote updates without integrating them

`git log` View commit history

`git log --oneline` View compact commit history

`git remote -v` Show configured remote URLs

`git diff` Show unstaged changes

`git diff --staged` Show staged changes

`git restore filename` Discard unstaged changes in a tracked file; use carefully

`git revert COMMIT_ID` Create a new commit that reverses a previous commit

Part 5: Quick Revision -- Important Differences

---------- Topic Main difference ---------- ---------- Git vs GitHub Git tracks versions; GitHub hosts Git repositories and supports collaboration.

Local vs Remote Repository Local is on your computer; remote is hosted elsewhere.

Add vs Commit Add stages changes; commit saves the staged snapshot.

Fetch vs Pull Fetch downloads updates; pull downloads and integrates them.

Merge vs Rebase Merge combines histories; rebase replays commits to create a linear history.

CI vs CD CI integrates and tests changes; CD prepares or automatically releases them.

Delivery vs Deployment Delivery may need manual production approval; deployment releases automatically after checks.

VM vs Container A VM runs a guest OS; a container shares the host OS kernel.

Part 6: Common Practical Workflow to Remember

Uploading changes to GitHub

```
git status
git add .
git commit -m "Describe your changes"
git push
```

Getting the latest changes

```
git pull
```

Creating a feature branch

```
git switch -c feature-name
```

Merging a feature branch into main

```
git switch main
git pull
git merge feature-name
git push
```

Checking repository history and remote

```
git log --oneline
git remote -v
```

Final Viva Tips

- Give a short definition first, then explain the purpose.

- For command-based questions, state the command and what it does.
- For comparison questions, explain both terms separately.
- Be ready to demonstrate `git status`, `git add`, `git commit`, `git branch`, `git push`, and `git pull`.
- Never share passwords, access tokens, or `.env` secrets in a public GitHub repository.
- Remember that GitHub is one hosting platform for Git; Git itself is not limited to GitHub.